

MINI-PROJECT 2

PRESENTATION

CSX4207 sec541



by Group 4



MINI-PROJECT 2 PRESENTATION

CSX4207 sec541



by Group 4

Thit Lwin Win Thant

6540122

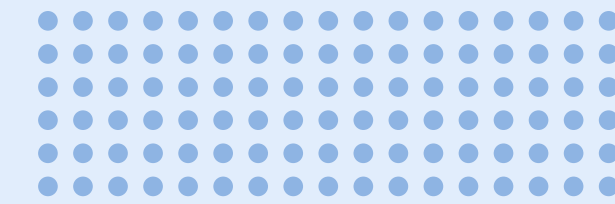
Kaung Khant Lin

6540131

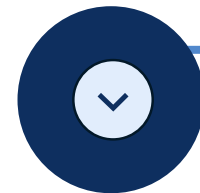
Thust Thongsricharoen

6714508

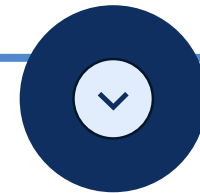




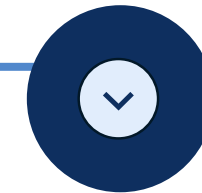
Contents



Introduction

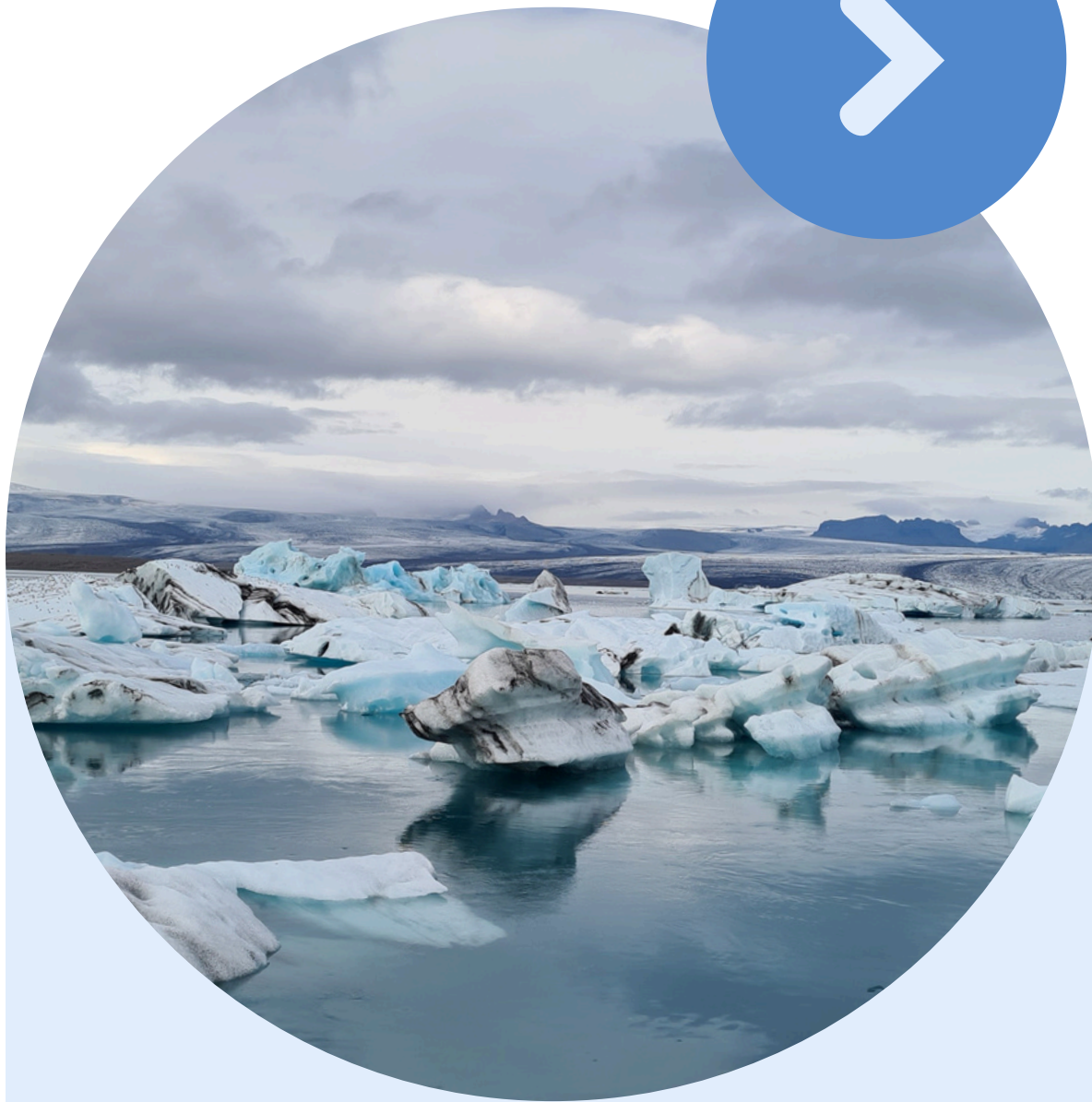


Objectives



References

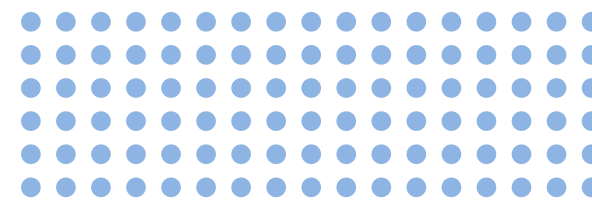




Introduction

This development focuses on the implementation and evaluation of a recommendation system algorithm using a prepared user book rating dataset.

- Two approaches:
 - User-based Nearest Neighbor recommender
 - Collaborative filtering approach





Objectives

Objectives

- Compute user similarities, predict unseen ratings and generate top book recommendations per user.
- The results are saved in structured CSV files, including similarity matrices and recommendation outputs.



Part 1:

A simple recommendation algorithm

Read a training set from the 'rating10user91_trainset.csv'

```
import pandas as pd

# Load the training and test datasets
train_df = pd.read_csv('rating10user91_trainset.csv')
test_df = pd.read_csv('rating10user91_testset.csv')

print("Training data loaded successfully.")
print(train_df.head())
print("\nTest data loaded successfully.")
print(test_df.head())
```

[1]

... Training data loaded successfully.

	userid	isbn	rating
0	6251	60392452	10
1	6251	61009059	7
2	6251	140067477	10
3	6251	375727345	6
4	6251	380789035	7

Test data loaded successfully.

	userid	isbn	rating
0	6251	312980140	7
1	6251	385484518	7

Read a test set from the 'rating10user91_testset.csv'

Part 1:

A simple recommendation algorithm

Get user Similarity Calculation by using Pearson Correlation

```
train_matrix = train_df.pivot_table(index='userid', columns='isbn', values='rating')

# --- User Similarity Calculation (Pearson Correlation) ---

# Function to calculate Pearson correlation between two users
def pearson_correlation(user_a_ratings, user_b_ratings):
    # Get common items rated by both users
    common_items = user_a_ratings.index.intersection(user_b_ratings.index)

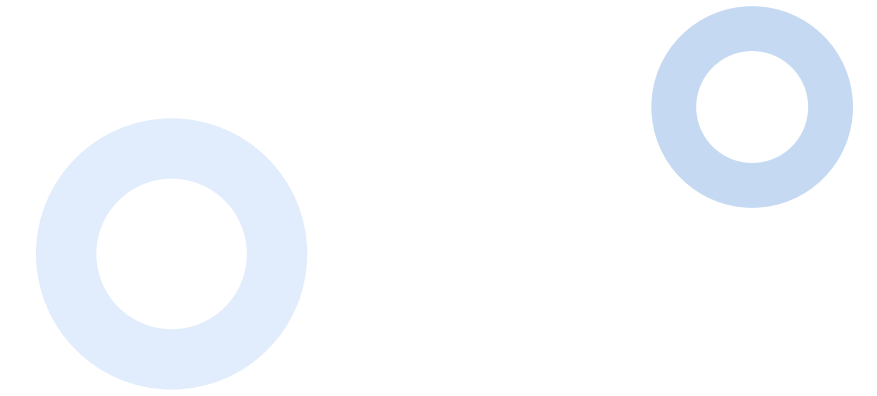
    # Filter ratings for common items
    ratings_a = user_a_ratings.loc[common_items]
    ratings_b = user_b_ratings.loc[common_items]

    # If there are no common items, similarity is 0
    if ratings_a.empty or ratings_b.empty:
        return 0

    # Calculate the mean of ratings for common items for each user
    mean_a = np.mean(ratings_a)
    mean_b = np.mean(ratings_b)
```

Part 1:

A simple recommendation algorithm



Pearson Correlation Continue

```
# Calculate the numerator (sum of product of differences from the mean)
numerator = np.sum((ratings_a - mean_a) * (ratings_b - mean_b))

# Calculate the denominator (product of standard deviations)
std_dev_a = np.sqrt(np.sum((ratings_a - mean_a)**2))
std_dev_b = np.sqrt(np.sum((ratings_b - mean_b)**2))

denominator = std_dev_a * std_dev_b

# Avoid division by zero
if denominator == 0:
    return 0
else:
    return numerator / denominator

# Create a DataFrame to store similarities
# Initialize with zeros
similarity_matrix = pd.DataFrame(index=train_matrix.index, columns=train_matrix.index, data=0.0)
```


Part 1:

A simple recommendation algorithm

Iterate over all unique user pairs to calculate similarity and save to a csv

```
users = train_matrix.index
for i in range(len(users)):
    for j in range(i + 1, len(users)):
        user1 = users[i]
        user2 = users[j]

        # Get the row for each user
        user1_ratings = train_matrix.loc[user1].dropna()
        user2_ratings = train_matrix.loc[user2].dropna()

        # Calculate similarity using only common items
        sim = pearson_correlation(user1_ratings, user2_ratings)

        similarity_matrix.loc[user1, user2] = sim
        similarity_matrix.loc[user2, user1] = sim # Similarity is symmetric

print("Similarity matrix calculated.")
print(similarity_matrix.head())

# Save the similarity matrix to a CSV file
similarity_matrix.to_csv('P2Part1_1PCC_Group4.csv')

print(f"User similarities (Pearson Correlation Coefficient) saved to 'P2Part1_1PCC_Group4.csv'")
```


Part 1:

A simple recommendation algorithm

Prepare the similarity matrix

```
# Load the similarity matrix
similarity_matrix = pd.read_csv('P2Part1_1PCC_Group4.csv', index_col='userid')
print("Similarity matrix loaded.")

# Convert userid columns to string type for consistent comparison
train_df['userid'] = train_df['userid'].astype(str)
test_df['userid'] = test_df['userid'].astype(str)

# Re-index similarity matrix with string user IDs if it's not already
similarity_matrix.index = similarity_matrix.index.astype(str)
similarity_matrix.columns = similarity_matrix.columns.astype(str)

# Create user-item matrix for training data
train_matrix = train_df.pivot_table(index='userid', columns='isbn', values='rating')
print(f"Shape of training matrix: {train_matrix.shape}")

# Create user-item matrix for test data
test_matrix = test_df.pivot_table(index='userid', columns='isbn', values='rating')
print(f"Shape of test matrix: {test_matrix.shape}")
```


Part 1:

A simple recommendation algorithm

Prediction Function

```
def predict_ratings(target_user_id, test_user_item_matrix, train_user_item_matrix, similarity_matrix, k=5):
    predictions = {}

    if target_user_id not in similarity_matrix.index:
        print(f"User {target_user_id} not found in similarity matrix index. Skipping predictions for this user.")
        return {}

    try:
        target_user_avg_rating = np.mean(train_user_item_matrix.loc[target_user_id].dropna())
    except KeyError:
        print(f"Error: Could not retrieve average rating for user {target_user_id} from training matrix.")
        return {}

    rated_items_by_target_user_in_train = train_user_item_matrix.loc[target_user_id].dropna().index.tolist()

    all_train_items = train_user_item_matrix.columns
    items_to_predict_for = [item for item in test_user_item_matrix.columns if item in all_train_items]
```

Part 1:

A simple recommendation algorithm

Prediction Function

```
for item_id in items_to_predict_for:
    if item_id not in rated_items_by_target_user_in_train:
        neighbor_data = []
        for neighbor_id in similarity_matrix.columns:
            if neighbor_id == target_user_id:
                continue

            if pd.isna(train_user_item_matrix.loc[neighbor_id, item_id]):
                try:
                    neighbor_avg_rating = np.mean(train_user_item_matrix.loc[neighbor_id].dropna())
                    sim_value = similarity_matrix.loc[target_user_id, neighbor_id] # Access similarity

                    neighbor_data.append({
                        'neighbor_id': neighbor_id,
                        'rating': train_user_item_matrix.loc[neighbor_id, item_id],
                        'avg_rating': neighbor_avg_rating,
                        'similarity': sim_value
                    })
                except KeyError:
                    print(f"KeyError accessing similarity for ({target_user_id}, {neighbor_id}) or training rating")
                    continue
            except Exception as e:
                print(f"An error occurred processing neighbor {neighbor_id} for item {item_id}: {e}")
                continue
```

Part 1:

A simple recommendation algorithm

Prediction Function

```
if neighbor_data:
    neighbor_df = pd.DataFrame(neighbor_data)
    top_k_neighbors = neighbor_df.nlargest(k, 'similarity')

    weighted_sum_numerator = 0
    sum_of_similarities = 0

    for _, row in top_k_neighbors.iterrows():
        neighbor_sim = row['similarity']
        neighbor_rating = row['rating']
        neighbor_avg = row['avg_rating']
        weighted_sum_numerator += neighbor_sim * (neighbor_rating - neighbor_avg)
        sum_of_similarities += neighbor_sim

    if sum_of_similarities != 0:
        predicted_rating = target_user_avg_rating + (weighted_sum_numerator / sum_of_similarities)
        predicted_rating = max(1, min(10, predicted_rating))
        predictions[(target_user_id, item_id)] = predicted_rating
    else:
        predictions[(target_user_id, item_id)] = target_user_avg_rating
else:
    predictions[(target_user_id, item_id)] = target_user_avg_rating

return predictions
```


Part 1:

A simple recommendation algorithm

Generate Predictions

```
# --- Generate Predictions ---
all_predictions = {}
k_value = 5
test_users = test_df['userid'].unique()

for user_id in test_users:
    if user_id in similarity_matrix.index:
        print(f"Predicting ratings for user: {user_id}")
        user_predictions = predict_ratings(
            user_id,
            test_matrix,
            train_matrix,
            similarity_matrix,
            k=k_value
        )
        all_predictions.update(user_predictions)
    else:
        print(f"User {user_id} not found in similarity matrix index. Skipping predictions for this user.")
```

Part 1:

A simple recommendation algorithm

Save Predictions to a CSV

```
# --- Format and Save Predictions ---
predictions_list = []
for (user, item), rating in all_predictions.items():
    predictions_list.append({'userid': user, 'isbn': item, 'predicted_rating': rating})

predicted_ratings_df = pd.DataFrame(predictions_list)

if not predicted_ratings_df.empty:
    predicted_ratings_df.to_csv('P2Part1_1PCC_Group4_predictions.csv', index=False)
    print("\nPredicted ratings for unseen items saved to 'P2Part1_1PCC_Group4_predictions.csv'")
    print(predicted_ratings_df.head())
else:
    print("\nNo predictions were generated.")
```

[illegible]

Part 1:

A simple recommendation algorithm

Import Predicted information

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# Load the data
df = pd.read_csv('P2Part1_1PCC_Group4_predictions.csv')

# Pivot to create user-item matrix
user_item_matrix = df.pivot_table(index='userid', columns='isbn', values='predicted_rating', fill_value=0)

# Compute cosine similarity between users
similarity_matrix = pd.DataFrame(cosine_similarity(user_item_matrix),
                                index=user_item_matrix.index,
                                columns=user_item_matrix.index)

# Function to get top 5 neighbors for a user
def get_top_neighbors(user_id, sim_matrix, top_n=5):
    neighbors = sim_matrix.loc[user_id].drop(user_id).sort_values(ascending=False).head(top_n)
    return neighbors.index.tolist()
```

Part 1:

A simple recommendation algorithm

Check Similarity between target user and other '1stNNUserID - 5NNUserID'

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# Load the data
df = pd.read_csv('P2Part1_1PCC_Group4_predictions.csv')

# Pivot to create user-item matrix
user_item_matrix = df.pivot_table(index='userid', columns='isbn', values='predicted_rating', fill_value=0)

# Compute cosine similarity between users
similarity_matrix = pd.DataFrame(cosine_similarity(user_item_matrix),
                                index=user_item_matrix.index,
                                columns=user_item_matrix.index)

# Function to get top 5 neighbors for a user
def get_top_neighbors(user_id, sim_matrix, top_n=5):
    neighbors = sim_matrix.loc[user_id].drop(user_id).sort_values(ascending=False).head(top_n)
    return neighbors.index.tolist()
```

Part 1:

A simple recommendation algorithm

Check Similarity between target user and other '1stNNUserID - 5NNUserID'

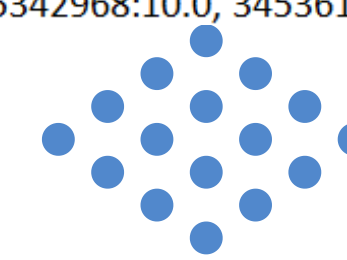
```
# Prepare final output
output_rows = []

for user_id in df['userid'].unique():
    neighbors = get_top_neighbors(user_id, similarity_matrix)
    top_books = df[df['userid'] == user_id].sort_values(by='predicted_rating', ascending=False).head(5)

    for _, row in top_books.iterrows():
        output_rows.append({
            'TargetUserID': user_id,
            '1stNNUserID': neighbors[0],
            '2NNUserID': neighbors[1],
            '3NNUserID': neighbors[2],
            '4NNUserID': neighbors[3],
            '5NNUserID': neighbors[4],
            'Book's ISBN': row['isbn'],
            'predicted rating': row['predicted_rating']
        })

# Convert to DataFrame
final_df = pd.DataFrame(output_rows)
final_df.to_csv('user_summary_top_books_and_neighbors.csv', index=False)
print(final_df)
```


	M	N
, 452282152:	9.95	
671001795:	7.73	
380718340:	10.0	
6027321:	8.7	
80813815:	10.0	
42001740:	9.45	
4023722X:	9.64	
43418174:	9.07	
45378490:	10.0	



Part 1:

A simple recommendation algorithm

'P2Part1_2Recommendation_Group4_ output

	A	B	C	D	E
1	TargetUserID	1stNNUserID	2ndNNUserID	3rdNNUserID	4thNNUserID
2	6251	53174	52584	56447	23872
3	6575	93047	88693	91113	53174
4	7346	91113	53174	101851	52584
5	11676	105979	56447	6251	68555
6	13552	17950	70666	53174	52584
7	16795	88693	53174	35857	52584
8	17950	53174	88693	93047	70666
9	21014	53174	105028	88693	52584
10	23872	53174	101851	93047	88693

Part 1:

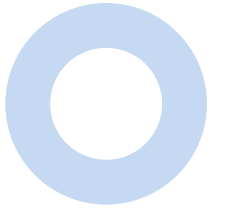
A simple recommendation algorithm

'P2Part1_2Recommendation_Group4_ output

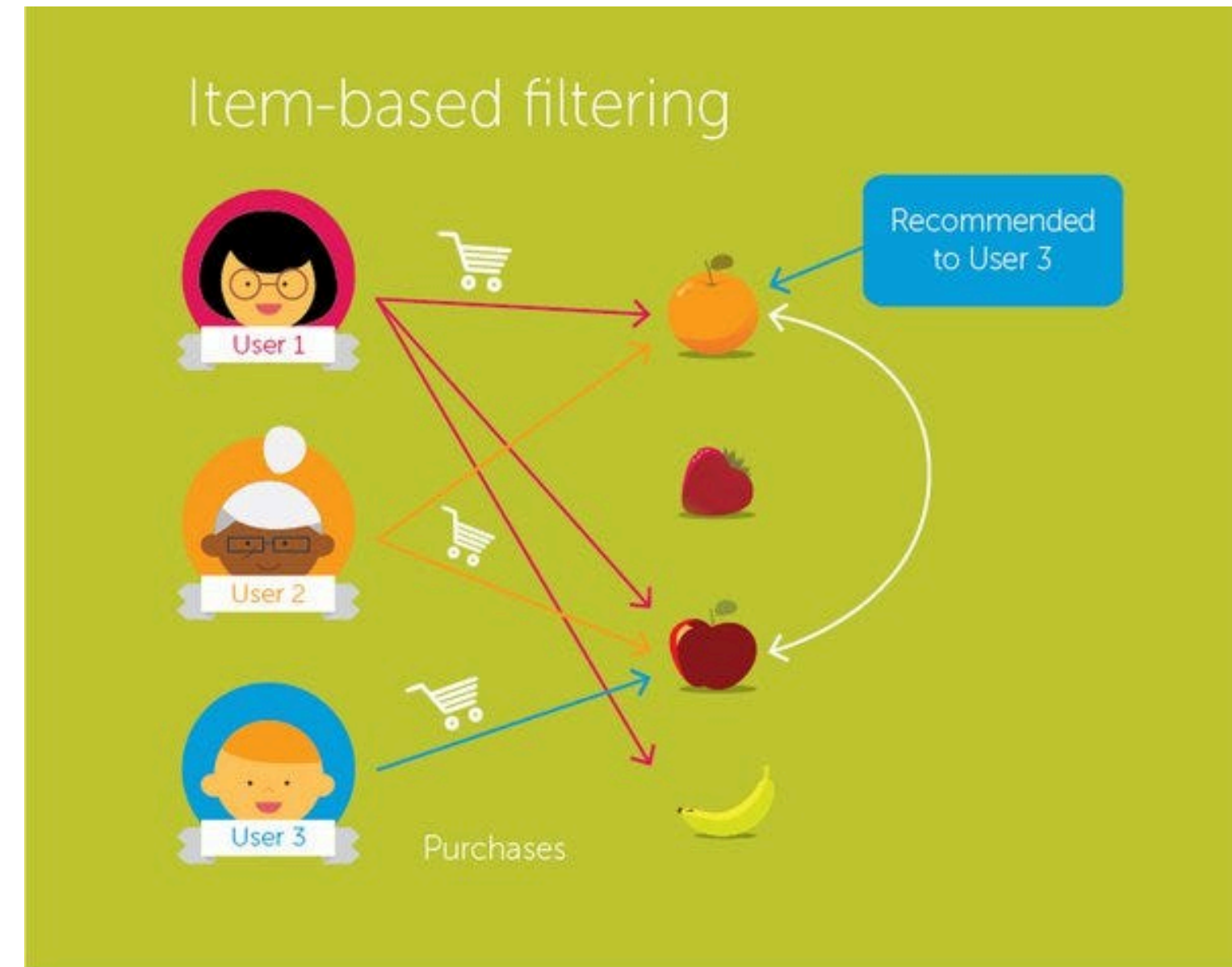
TopBooks (ISBN:Rating)					
080410526X:10.0, 743418174:10.0, 044651652X:10.0, 446605239:10.0, 452282152:9.95					
1558744150:8.59, 380718340:8.59, 142001740:8.36, 446310786:7.88, 671001795:7.73					
440998050:10.0, 553250426:10.0, 385484518:10.0, 044651652X:10.0, 380718340:10.0					
440206154:10.0, 60938455:10.0, 446310786:9.26, 312980140:9.09, 156027321:8.7					
385722206:10.0, 142001740:10.0, 446605239:10.0, 446310786:10.0, 380813815:10.0					
440234743:9.95, 439064864:9.86, 385484518:9.82, 345361792:9.62, 142001740:9.45					
380018179:9.9, 446310786:9.9, 316666343:9.64, 043935806X:9.64, 044023722X:9.64					
044651652X:10.0, 345350499:9.41, 345313860:9.36, 312980140:9.1, 743418174:9.07					
399501487:10.0, 316776963:10.0, 345342968:10.0, 345361792:10.0, 345378490:10.0					

Part 2

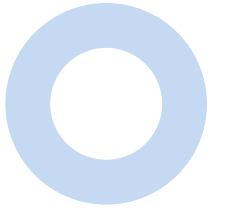
A simple collaborative based filtering algorithm



We chose to use Item-base collaborative filtering
(Lecture 5)



Part 2

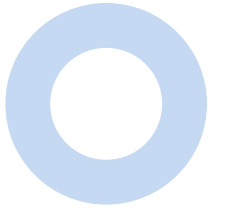


A simple collaborative based filtering algorithm

Data Preparation (Mean-Centering)

- Load the training ratings (../rating10user91_trainset.csv).
- Convert IDs to strings, ratings to numeric, drop NaNs.
- Compute each user's average rating and subtract it from their ratings (mean-centering).
- Build a user-item pivot table of centered ratings and save it.

Part 2



A simple collaborative based filtering algorithm

1. Load and clean the training data

```
import pandas as pd
df = pd.read_csv("../rating10user91_trainset.csv")
df['user'] = df['user'].astype(str)
df['item'] = df['item'].astype(str)
df['rating'] = pd.to_numeric(df['rating'], errors='coerce')
df = df.dropna(subset=['rating']).copy()
```

2. Compute each user's mean rating

```
user_mean = df.groupby('user')['rating'].mean()
df['user_mean'] = df['user'].map(user_mean)
```

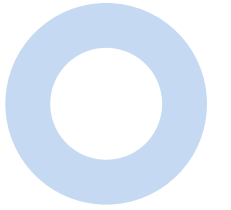
3. Create the mean-centered rating column

```
df['rating_centered'] = df['rating'] - df['user_mean']
```

4. Create the user-item matrix from centered ratings

```
pivot = df.pivot_table(index='user', columns='item', values='rating_centered', aggfunc='mean')
pivot.to_csv("P2Part2_1Profile_Group4.csv", float_format='%.4f', na_rep='')
```

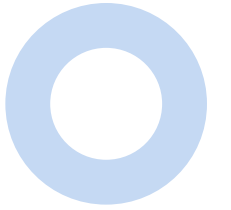
Part 2



A simple collaborative based filtering algorithm

	A	B	C	D	E	F	G	H	I	J	K	L
1	user	014028009X	014029628X	034538475X	043935806X	044021145X	044022165X	044023722X	044651652X	059035342X	067976402X	068484X
2	Possible Data Loss				1							
3	102647											
4	104636					1.375	-0.625			1.375		
5	105028											
6	105517				-2.4286							
7	105979					1.7857	-1.2143					
8	107784			-2.4615		0.5385	0.5385			-2.4615		
9	110912	0.5238	-0.4762									
10	110934											
11	113270										1.4	
12	11676	0.3714	-0.6286	-1.6286		-6.6286		0.3714	0.3714	2.3714	2.3714	
13	126492			-0.9444		0.0556						
14	135149				1			0				
15	13552						-0.7		-0.7			
16	136382											
17	140000											
18	140358			0		0	0				0	
19	147847					-0.8						
20	148744					1.4545						
21	150979						-1.0833					
22	153662				0.5714					0.5714	0.5714	
23	158295					-0.7	-0.7					
24	158433					-0.75						
25	159506				0.3077	0.3077				-0.6923		

Part 2



A simple collaborative based filtering algorithm

Model Building — Adjusted Cosine Similarity

- Use the mean-centered pivot table.
- For each item pair, find users who rated both.
- Compute numerator and denominator as above.
- Require at least 2 co-raters (MIN_OVERLAP = 2) for stability.
- Save the item-item similarity matrix (P2Part2_2Model_Group4.csv).

$$sim(a, b) = \frac{\sum_{u \in U} (r_{u,a} - \bar{r}_u)(r_{u,b} - \bar{r}_u)}{\sqrt{\sum_{u \in U} (r_{u,a} - \bar{r}_u)^2} \sqrt{\sum_{u \in U} (r_{u,b} - \bar{r}_u)^2}}$$

Part 2

A simple collaborative based filtering algorithm

```
import numpy as np
import pandas as pd

# Parameters from the notebook
MIN_OVERLAP = 2
APPLY_SHRINKAGE = False
SHRINKAGE_LAMBDA = 10

# Get items and values from the centered pivot table
centered_pivot = pivot
items = centered_pivot.columns.to_list()
n_items = len(items)
values = centered_pivot.values
sim_mat = np.zeros((n_items, n_items))

# Loop through each unique pair of items
for i in range(n_items):
    vi = values[:, i]
    mask_i = ~np.isnan(vi)
    for j in range(i, n_items):
        if i == j:
            sim_mat[i, i] = 1.0
            continue

        vj = values[:, j]
        mask_j = ~np.isnan(vj)

        # Find users who rated both items
        common = mask_i & mask_j
        c = int(common.sum())

        # Skip if not enough co-raters
        if c < MIN_OVERLAP:
            continue
```

Part 2

A simple collaborative based filtering algorithm

```
# Get centered ratings from common users
vi_c = vi[common]
vj_c = vj[common]

# Calculate dot product (numerator) and vector norms (denominator)
num = np.dot(vi_c, vj_c)
denom_i = np.sqrt(np.dot(vi_c, vi_c))
denom_j = np.sqrt(np.dot(vj_c, vj_c))

if denom_i == 0 or denom_j == 0:
    continue

raw_sim = num / (denom_i * denom_j)

# Apply optional shrinkage (disabled in our final run)
sim_val = raw_sim
if APPLY_SHRINKAGE:
    weight = c / (c + SHRINKAGE_LAMBDA)
    sim_val = weight * raw_sim

sim_mat[i, j] = sim_val
sim_mat[j, i] = sim_val

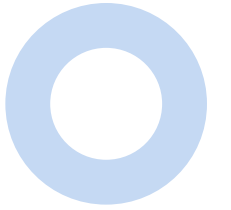
# Save the final similarity matrix to a file
sim_df = pd.DataFrame(sim_mat, index=items, columns=items)
sim_df.to_csv("P2Part2_2Model_Group4.csv", float_format="%.6f")
```

Part 2

A simple collaborative based filtering algorithm

	A	B	C	D	E	F	G	H	I	J	
1		014028009X	014029628X	034538475X	043935806X	044021145X	044022165X	044023722X	044651652X	059035342X	0
2	014028009X	1	0.090457	0	-0.955511	-0.321353	0	-0.188963	-0.558852	0.193226	
3	014029628X	0.090457	1	-0.263681	0	0	0	0.116626	0	0	
4	034538475X	0	-0.263681	1	0	0.382436	-1	0	0	0.21779	
5	043935806X	-0.955511	0	0	1	0.612487	0	0.906183	0	-0.52498	
6	044021145X	-0.321353	0	0.382436	0.612487	1	-0.515565	-0.031152	-0.709758	-0.434953	
7	044022165X	0	0	-1	0	-0.515565	1	0	-0.76016	-0.942139	
8	044023722X	-0.188963	0.116626	0	0.906183	-0.031152	0	1	-0.22124	0.725033	
9	044651652X	-0.558852	0	0	0	-0.709758	-0.76016	-0.22124	1	0	
10	059035342X	0.193226	0	0.21779	-0.52498	-0.434953	-0.942139	0.725033	0	1	
11	067976402X	0.721767	0	-0.316947	0.921894	-0.496349	1	0	0	-0.158932	
12	068484267X	0	0	0	0	-0.145114	0	0	0	0	
13	068484477X	-0.455946	0.702638	0	-0.906183	0.927136	0	-0.392728	-0.364244	-0.649144	
14	080410526X	-0.724878	0.267924	0.858698	0	0.55672	0	0	0	0.060865	
15	1400031354	0.379724	-0.953648	0	0	-0.468539	0	0	0	0	
16	1400034779	-0.766319	0.691244	0	0	-0.284634	0	0	0	0	
17	140067477	-0.279526	-0.483858	-0.171806	1	-0.709758	-0.301892	0	0	0.728746	
18	140298479	0.356205	0	0	0	0.648105	0	-0.762561	0	-0.990403	
19	142001740	0	0.799813	0	0	-0.942361	0	0	0	0	
20	1558744150	0	0	0	0	-0.711093	0.104883	0	0.439063	0.032358	
21	156027321	0	0	0	0	0	0	0	0	0	
22	312195516	-0.925619	-0.357221	0	0.056759	0	0	0.774546	0	0	
23	312278586	0.073456	0	0.93955	0.711294	0.73043	0	0	0	-0.864021	
24	312924585	-0.781853	0	-0.754763	-0.13675	-0.1937	-0.815567	0.676584	0	-0.688154	

Part 2



A simple collaborative based filtering algorithm

Online Prediction — Weighted Sum over Similar Items

- For each item, precompute a list of top-K most similar neighbors (positive similarity only, TOP_K_NEIGHBORS = 10).
- For each user, for each candidate item, accumulate numerator and denominator from rated items' neighbors.
- If no contributing neighbors, fall back to user mean (or global mean for cold-start).
- Sort candidates by predicted score and write top-N recommendations per user (P2Part2_3Recommendation_Group4.csv).

$$pred(u, p) = \frac{\sum_{i \in ratedItems(u)} \overbrace{sim(i, p) \times r_{u,i}}}{\sum_{i \in ratedItems(u)} sim(i, p)}$$

Part 2

A simple collaborative based filtering algorithm

```
def predict_for_user(user_id):
    rated_pairs = user_ratings.get(user_id, [])
    if not rated_pairs:
        return [] # Cannot predict for a user with no ratings

    rated_items = {it for it, _ in rated_pairs}
    candidates = all_items - rated_items

    num = {} # Numerator accumulator for each candidate item
    denom = {} # Denominator accumulator for each candidate item

    # For each item the user has rated...
    for i, r_ui in rated_pairs:
        # Get its pre-computed neighbors
        for j, sim_val in item_neighbors.get(i, []):
            if j in candidates:
                # Accumulate the numerator: sim(i, j) * rating(u, i)
                num[j] = num.get(j, 0.0) + sim_val * r_ui
                # Accumulate the denominator: sim(i, j)
                denom[j] = denom.get(j, 0.0) + sim_val

    # Calculate final predictions
    preds = []
    u_mean = user_mean.get(user_id, global_mean) # Fallback rating
    for j in candidates:
        if j in num and denom.get(j, 0) != 0:
            pred = num[j] / denom[j]
        else:
            pred = u_mean # Fallback if no neighbors contributed

        # Clip predictions to the valid rating scale (e.g., 1-10)
        preds.append((j, max(CLIP_MIN, min(CLIP_MAX, pred))))

    preds.sort(key=lambda x: (-x[1], x[0]))
    return preds
```

Part 2

A simple collaborative based filtering algorithm

```
target_users = sorted(test_df['user'].unique())

records = []
for u in target_users:
    pred_list = predict_for_user(u)
    top_k = pred_list[:TOP_N]
    for item, score in top_k:
        records.append((u, item, score))

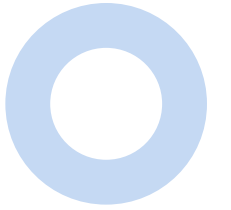
recs_df = pd.DataFrame(records, columns=['user', 'item', 'pred_rating'])
recs_df.to_csv(RECOMMEND_PATH, index=False, float_format="%.4f")
```

Part 2

A simple collaborative
based filtering
algorithm

	A	B	C	D
1	user	item	pred_rating	
2	101851	014028009X	10	
3	101851	067976402X	10	
4	101851	1400031354	10	
5	101851	312278586	10	
6	101851	312983271	10	
7	101851	316096199	10	
8	101851	316776963	10	
9	101851	345350499	10	
10	101851	380018179	10	
11	101851	385484518	10	
12	102647	068484267X	10	
13	102647	080410526X	10	
14	102647	112222122	10	

Part 2



A simple collaborative based filtering algorithm

RMSE Evaluation

- For each test row (user, item, actual), use `predict_for_user(user)` to get predictions.
- If no prediction for an item, fall back to user mean.
- Save all rows and report RMSE plus fallback counts (P2Part2_4_RMSE_Group4.csv).

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2}$$

Part 2

A simple collaborative based filtering algorithm

```
from math import sqrt

# Get predictions for all users in the test set
test_users = test_df['user'].unique()
all_user_preds = {u: dict(predict_for_user(u)) for u in test_users}

records = []
fallback_count = 0

# For each actual rating in the test data...
for _, row in test_df.iterrows():
    user, item, actual_rating = str(row['user']), str(row['item']), float(row['rating'])

    # Look up the pre-computed prediction for this user
    predicted_rating = all_user_preds.get(user, {}).get(item)

    # If no prediction, use the user's average rating as a fallback
    if predicted_rating is None:
        predicted_rating = user_mean.get(user, global_mean)
        fallback_count += 1

    records.append((user, item, actual_rating, predicted_rating))

# Create a DataFrame and compute RMSE
pred_df = pd.DataFrame(records, columns=["user", "item", "actual_rating", "predicted_rating"])
mse = ((pred_df["actual_rating"] - pred_df["predicted_rating"]) ** 2).mean()
rmse = sqrt(mse)

print(f"Final RMSE on test set: {rmse:.4f}")
print(f"Number of fallback predictions: {fallback_count}")
```

Part 2

A simple collaborative based filtering algorithm

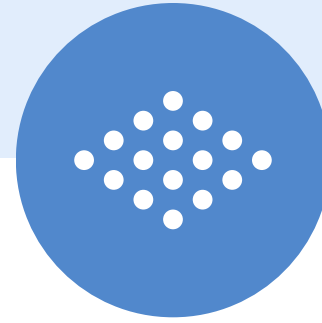
	A	B	
1	RMSE	1.7274	
2			
3			
4			
5			

References



- **Lecture Slides - Week 5 and 6 pdfs**
- **<https://www.geeksforgeeks.org/python/python-pearson-correlation-test-between-two-variables/>**





**THANK
YOU!**

